Title of the project :-

Implement and Manage Zero Trust Network Access (ZTNA) Using Zscaler.

Overview :-

The project aims to develop an AI-driven identity verification system that enhances online security by analysing and verifying user identities based on their online behaviour patterns. This system uses advanced machine learning techniques to create a robust user profile, distinguishing between legitimate users and suspicious activities. It addresses the pressing issue of unauthorised access, fraudulent transactions, and cyber threats across various online platforms and services. The primary objective is to enhance online security significantly by scrutinising and validating user identities through their online behaviour. The system employs advanced machine learning techniques such as deep learning, neural networks, and natural language processing to establish comprehensive behavioural profiles for each user, which adapt and evolve with the user's behaviour over time. Real-time anomaly detection is a key feature of the system, identifying deviations from established user behaviour profiles and responding promptly to mitigate potential security threats. Multi-factor verification is also implemented to bolster security, triggering additional verification steps when anomalies are detected. User and administrator alerts ensure transparent communication and

provide guidance on resolving potential security issues. Secure data storage and management components are included to protect sensitive user profiles and anomaly data. The system includes an integration layer for seamless interaction with various online platforms and services. Monitoring and reporting are integrated for performance and security, providing transparency and accountability. Scalability and load balancing are designed to handle growing user bases, while security and compliance are prioritised by encrypting data in transit and at rest, employing firewalls, intrusion detection systems, and adhering to data protection and privacy regulations. Cloud infrastructure is an optional option for flexibility and scalability. Continuous training and optimization of machine learning models ensure the system remains effective against evolving threats and user behaviours. Geographic distribution can enhance redundancy and performance by deploying components in multiple geographic regions. Multi-factor verification is also implemented to bolster security, triggering additional verification steps when anomalies are detected. User and administrator alerts ensure transparent communication and provide guidance on resolving potential security issues. Secure data storage and management components are included to protect sensitive user profiles and anomaly data. The system includes an integration layer for seamless interaction with various online platforms and services. Monitoring and reporting are integrated for performance and security, providing transparency and accountability. Scalability

and load balancing are designed to handle growing user bases, while security and compliance are prioritised by encrypting data in transit and at rest, employing firewalls, intrusion detection systems, and adhering to data protection and privacy regulations. Cloud infrastructure is an optional option for flexibility and scalability. Continuous training and optimization of machine learning models ensure the system remains effective against evolving threats and user behaviours. Geographic distribution can enhance redundancy and performance by deploying components in multiple geographic

List of teammates–

S.no	**Name**	**College**	**Mail id**
1.	Brinda .P.A	SVASC, Gobi	brindapa66@gmail.com
2.	Nandhini .K	SVASC, Gobi	nandhinik2023aids@gmail.com
3.	Janani .L.P	SVASC, Gobi	jananilp2023aids@gmail.com
4.	Ayeshasithiha .J	SVASC, Gobi	ayeshasithihaj2023aids@gmail.com
5.	Fathima .K	SVASC, Gobi	fathimak2023aids@gmail.com

List of Vulnerability Table —

S.n	Category Name o	Vulnerability name	CWE/CVE - No
1.	Broken Access Control	Incorrect Default Permissions	CWE-276
2.	Security Misconfiguration	External Control of System or Configuration Setting	CWE-15
3.	Software Supply Chain Failures	Dependency on Vulnerable Third-Party Component	CWE-1395

REPORT:-

1.

Vulnerability Name:- Improper Limitation of a Pathname to a Restricted Directory ('Path Traversal')

CWE/CVE : -CWE-22

OWASP/SANS Category:- A01:2025 Broken Access Control

Description:-

Access control enforces policy such that users cannot act outside of their intended permissions. Failures typically lead to unauthorized information disclosure, modification or destruction of all data, or performing a business function outside the user's limits. Common access control vulnerabilities include:

- Violation of the principle of least privilege, commonly known as deny by default, where access should only be granted for particular capabilities, roles, or users, but is available to anyone.
- Bypassing access control checks by modifying the URL (parameter tampering or force browsing), internal application state, or the HTML page, or by using an attack tool that modifies API requests.
- Permitting viewing or editing someone else's account by providing its unique identifier (insecure direct object references)
- An accessible API with missing access controls for POST, PUT, and DELETE.
- Elevation of privilege. Acting as a user without being logged in or or gaining privileges beyond those expected of the logged in user (e.g. admin access).
- Metadata manipulation, such as replaying or tampering with a JSON Web Token (JWT) access

control token, a cookie or hidden field manipulated to elevate privileges, or abusing JWT invalidation.

- CORS misconfiguration allows API access from unauthorized or untrusted origins.
- Force browsing (guessing URLs) to authenticated pages as an unauthenticated user or to privileged pages as a standard user.

Business Impact::-

Access control is only effective when implemented in trusted server-side code or serverless APIs, where the attacker cannot modify the access control check or metadata.

- Except for public resources, deny by default.
- Implement access control mechanisms once and reuse them throughout the application, including minimizing Cross-Origin Resource Sharing (CORS) usage.
- Model access controls should enforce record ownership rather than allowing users to create, read, update, or delete any record.
- Unique application business limit requirements should be enforced by domain models.

- Disable web server directory listing and ensure file metadata (e.g., .git) and backup files are not present within web roots.
- Log access control failures, alert admins when appropriate (e.g., repeated failures).
- Implement rate limits on API and controller access to minimize the harm from automated attack tooling.
- Stateful session identifiers should be invalidated on the server after logout. Stateless JWT tokens should be short-lived to minimize the window of opportunity for an attacker. For longer-lived JWTs, consider using refresh tokens and following OAuth standards to revoke access.
- Use well-established toolkits or patterns that provide simple, declarative access controls.

Developers and QA staff should include functional access control in their unit and integration tests.

2.

Vulnerability Name:- External Control of System or Configuration Setting

CWE/CVE : - CWE-15

OWASP/SANS Category:- A02:2025 Security Misconfiguration

Description:-

Security misconfiguration is when a system, application, or cloud service is set up incorrectly from a security perspective, creating vulnerabilities.

The application might be vulnerable if:

- It is missing appropriate security hardening across any part of the application stack or improperly configured permissions on cloud services.
- Unnecessary features are enabled or installed (e.g., unnecessary ports, services, pages, accounts, testing frameworks, or privileges).
- Default accounts and their passwords are still enabled and unchanged.
- A lack of central configuration for intercepting excessive error messages. Error handling reveals stack traces or other overly informative error messages to users.
- For upgraded systems, the latest security features are disabled or not configured securely.
- Excessive prioritization of backward compatibility leading to insecure configuration.

- The security settings in the application servers, application frameworks (e.g., Struts, Spring, ASP.NET), libraries, databases, etc., are not set to secure values.
- The server does not send security headers or directives, or they are not set to secure values.

Without a concerted, repeatable application security configuration hardening process, systems are at a higher risk.

Business Impact::-

Secure installation processes should be implemented, including:

- A repeatable hardening process enabling the fast and easy deployment of another environment that is appropriately locked down. Development, QA, and production environments should all be configured identically, with different credentials used in each environment. This process should be automated to minimize the effort required to set up a new secure environment.
- A minimal platform without any unnecessary features, components, documentation, or samples. Remove or do not install unused features and frameworks.

- A task to review and update the configurations appropriate to all security notes, updates, and patches as part of the patch management process (see A03 Software Supply Chain Failures). Review cloud storage permissions (e.g., S3 bucket permissions).
- A segmented application architecture provides effective and secure separation between components or tenants, with segmentation, containerization, or cloud security groups (ACLs).
- Sending security directives to clients, e.g., Security Headers.
- An automated process to verify the effectiveness of the configurations and settings in all environments.
- Proactively add a central configuration to intercept excessive error messages as a backup.
- If these verifications are not automated, they should be manually verified annually at a minimum.
- Use identity federation, short-lived credentials, or role-based access mechanisms provided by the underlying platform instead of embedding static keys or secrets in code, configuration files, or pipelines.

3.

Vulnerability Name:- Dependency on Vulnerable Third-Party Component

CWE/CVE : -CWE-1395

OWASP/SANS Category:- A03:2025 Security Misconfiguration

Description:-

Software supply chain failures are breakdowns or other compromises in the process of building, distributing, or updating software. They are often caused by vulnerabilities or malicious changes in third-party code, tools, or other dependencies that the system relies on.

You are likely vulnerable if:

- you do not carefully track the versions of all components that you use (both client-side and server-side). This includes components you directly use as well as nested (transitive) dependencies.
- the software is vulnerable, unsupported, or out of date. This includes the OS, web/application server, database management system (DBMS), applications, APIs and all components, runtime environments, and libraries.

- you do not scan for vulnerabilities regularly and subscribe to security bulletins related to the components you use.
- you do not have a change management process or tracking of changes within your supply chain, including tracking IDEs, IDE extensions and updates, changes to your organization's code repository, sandboxes, image and library repositories, the way artifacts are created and stored, etc. Every part of your supply chain should be documented, especially changes.
- you have not hardened every part of your supply chain, with a special focus on access control and the application of least privilege.
- your supply chain systems do not have any separation of duty. No single person should be able to write code and promote it all the way to production without oversight from another human being.
- components from untrusted sources, across any part of the tech stack, are used in or can impact on production environments.
- you do not fix or upgrade the underlying platform, frameworks, and dependencies in a risk-based, timely fashion. This commonly happens in environments when patching is a monthly or quarterly task under change control, leaving organizations open to days or

months of unnecessary exposure before fixing vulnerabilities.

- software developers do not test the compatibility of updated, upgraded, or patched libraries.
- you do not secure the configurations of every part of your system (see A02:2025-Security Misconfiguration).
- your CI/CD pipeline has weaker security than the systems it builds and deploys, especially if it is complex.

Business Impact::-

There should be a patch management process in place to:

- Centrally generate and manage the Software Bill of Materials (SBOM) of your entire software.
- Track not just your direct dependencies, but their (transitive) dependencies, and so on.
- Reduce attack surface by removing unused dependencies, unnecessary features, components, files, and documentation.
- Continuously inventory the versions of both client-side and server-side components (e.g., frameworks, libraries) and their dependencies using tools like

OWASP Dependency Track, OWASP Dependency Check, retire.js, etc.

- Continuously monitor sources like Common Vulnerability and Exposures (CVE), National Vulnerability Database (NVD), and OpenSource Vulnerabilities (OSV) for vulnerabilities in the components you use. Use software composition analysis, software supply chain, or security-focused SBOM tools to automate the process. Subscribe to alerts for security vulnerabilities related to components you use.
- Only obtain components from official (trusted) sources over secure links. Prefer signed packages to reduce the chance of including a modified, malicious component (see A08:2025-Software and Data Integrity Failures).
- Deliberately choose which version of a dependency you use and upgrade only when there is need.
- Monitor for libraries and components that are unmaintained or do not create security patches for older versions. If patching is not possible, consider migrating to an alternative. If that is not possible, consider deploying a virtual patch to monitor, detect, or protect against the discovered issue.
- Update your CI/CD, IDE, and any other developer tooling regularly

- Avoid deploying updates to all systems simultaneously. Use staged rollouts or canary deployments to limit exposure in case a trusted vendor is compromised.

There should be a change management process or tracking system in place to track changes to:

- CI/CD settings (all build tools and pipelines)
- Code repositories
- Sandbox areas
- Developer IDEs
- SBOM tooling, and created artifacts
- Logging systems and logs
- Third party integrations, such as SaaS
- Artifact repositories
- Container registries

Harden the following systems, which includes enabling MFA and locking down IAM:

- Your code repository (which includes not checking in secrets, protecting branches, backups)
- Developer workstations (regular patching, MFA, monitoring, and more)

- Your build server & CI/CD (separation of duties, access control, signed builds, environment-scoped secrets, tamper-evident logs, more)
- Your artifacts (ensure integrity via provenance, signing, and time stamping, promote artifacts rather than rebuilding for each environment, ensure builds are immutable)
- Infrastructure as code (managed like all code, including use of PRs and version control)

Every organization must ensure an ongoing plan for monitoring, triaging, and applying updates or configuration changes for the lifetime of the application or portfolio.